Implementation of FIR Filters in Verilog

1 Introduction

Digital filters are widely used in signal processing to remove unwanted frequencies or to extract useful information from signals. One important class of digital filters is the **Finite Impulse Response (FIR) filter**.

In this project we implemented a 100-tap FIR filter in Verilog using three different methods:

- Direct Form FIR implementation
- FIR implementation using `generate` and `genvar`
- Optimized FIR implementation using coefficient symmetry

The goal of this project is to understand how FIR filters work and how they can be implemented efficiently in hardware.

2 Theory

2.1 Finite Impulse Response (FIR) Filters

A Finite Impulse Response (FIR) filter is a type of digital filter whose impulse response has a finite duration. The output depends only on the current and a finite number of past input samples.

The input-output relation is

$$y[n] = \sum_{k=0}^{N-1} h[k]\, x[n-k] \tag{1}$$

where

- $x[n]$ is the input signal
- $y[n]$ is the output signal
- $h[k]$ are filter coefficients
- N is number of taps

2.2 Convolution Interpretation

FIR filtering can also be interpreted as convolution:

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] \tag{2}$$

Since FIR filters have finite impulse response,

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k] \tag{3}$$

3 Direct Form FIR Implementation

In the direct form implementation, the input samples are stored in a shift register. Each delayed sample is multiplied with its corresponding coefficient and all the products are added together.

The structure follows the FIR equation directly.

3.1 Working Principle

The operation in each clock cycle is:

1. A new input sample enters the filter.
2. Previous samples shift through the delay line.
3. Each sample is multiplied by a filter coefficient.
4. All products are accumulated to produce the output.

This method is straightforward but requires a large number of multipliers.
For a 100-tap filter:

- 100 multipliers
- 99 adders
- 100 registers

3.2 Example Code Snippet

```
for (k = 0; k < N; k = k+1) begin
    prod = shift_reg[k] * h[k];
    acc  = acc + prod;
end
```

This loop multiplies each stored sample with the corresponding coefficient and adds the result to the accumulator.

4 FIR Implementation Using Generate and Genvar

The second method uses a modular approach. Instead of writing the complete computation inside one block, the filter is built using smaller modules called **Filter Fundamental Blocks (FFB)**.

Each block performs the following operations:

- Registers the input signal (delay element)
- Multiplies the signal with the coefficient
- Adds the result to the running sum

Multiple FFB blocks are connected in series to form the full filter.

4.1 Generate Block

The `generate` statement is used to automatically create multiple FFB blocks.

```
genvar i;
generate
for (i=0;i<N;i=i+1)
begin
    FFB tap(
        a_chain[i],
        b_chain[i],
        h[i],
        a_chain[i+1],
        b_chain[i+1]
    );
end
endgenerate
```

This makes the code modular and easier to scale when the number of taps increases.

5 Optimized FIR Implementation

Many FIR filters used in practice have **linear phase**. Such filters have symmetric coefficients:

$$h[k] = h[N-1-k]$$

Using this property we can rewrite the FIR equation as

$$y[n] = h_0(x[n] + x[n-99]) + h_1(x[n-1] + x[n-98]) + \cdots$$

Instead of multiplying each sample separately, symmetric input samples are first added together and then multiplied with a single coefficient.

5.1 Advantages

This optimization reduces the number of multipliers by half.

For a 100-tap FIR filter:

- Direct form: 100 multipliers
- Optimized form: 50 multipliers

This significantly reduces hardware cost.

5.2 Implementation Steps

Each clock cycle performs the following operations:

1. Insert a new input sample into the shift register.
2. Form symmetric pairs of input samples.
3. Add each pair.
4. Multiply the result with the corresponding coefficient.
5. Accumulate all products to produce the output.

5.3 Example Code Snippet

```
sym_sum = shift_reg[k] + shift_reg[N-1-k];
prod = sym_sum * h[k];
acc = acc + prod;
```

6 Testbench and Verification

To verify the FIR filter implementations, a Verilog testbench was developed.

The verification flow is as follows:

1. MATLAB generates the input signals and filter coefficients.
2. The testbench reads these values from text files.
3. Input samples are applied to the FIR filter one per clock cycle.
4. The filter output is written to output files.
5. MATLAB reads the output files and compares them with the expected results.

Three different test signals were used:

- 950 Hz signal
- 1100 Hz signal
- 2000 Hz signal

The outputs from the FIR filter were compared with MATLAB results to ensure correct functionality.

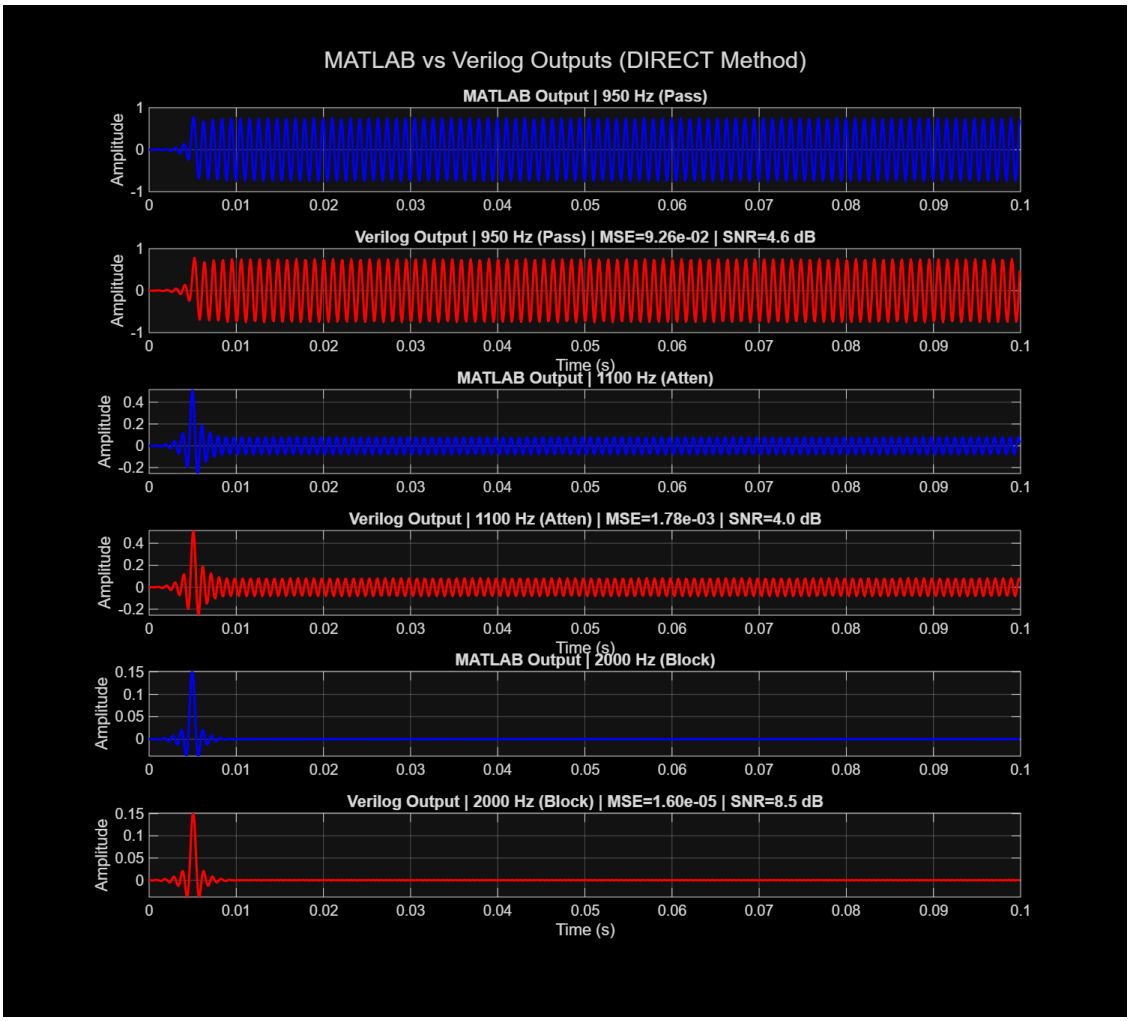

Figure 1: MATLAB vs Verilog comparison for Direct Form FIR filter implementation. The 950 Hz signal passes through the filter, while the 1100 Hz and 2000 Hz signals are attenuated as expected for a low-pass filter with a 1000 Hz cutoff.

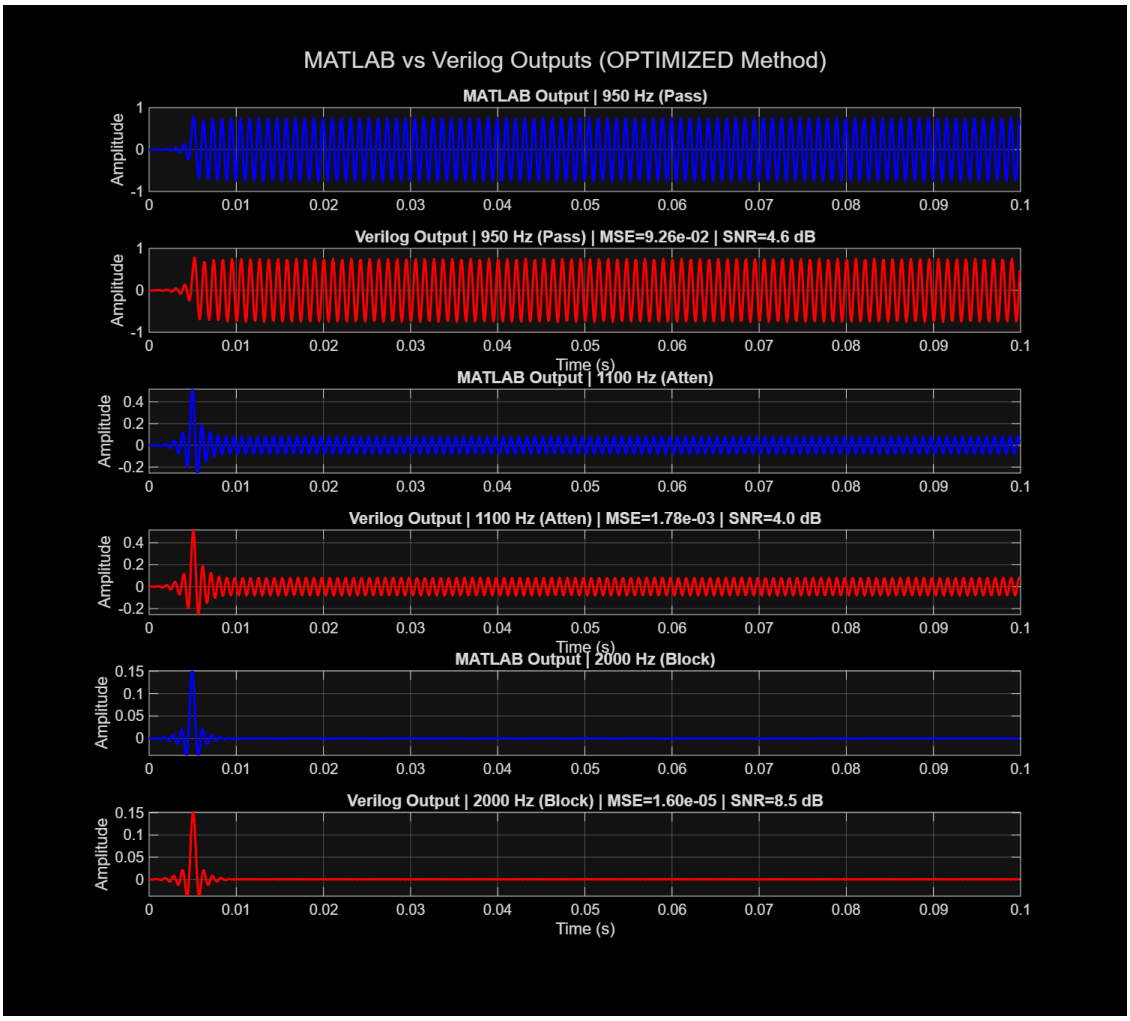

Figure 2: MATLAB vs Verilog comparison for the optimized symmetric FIR implementation. The symmetric property of FIR coefficients reduces the number of multipliers while maintaining the same filtering performance.

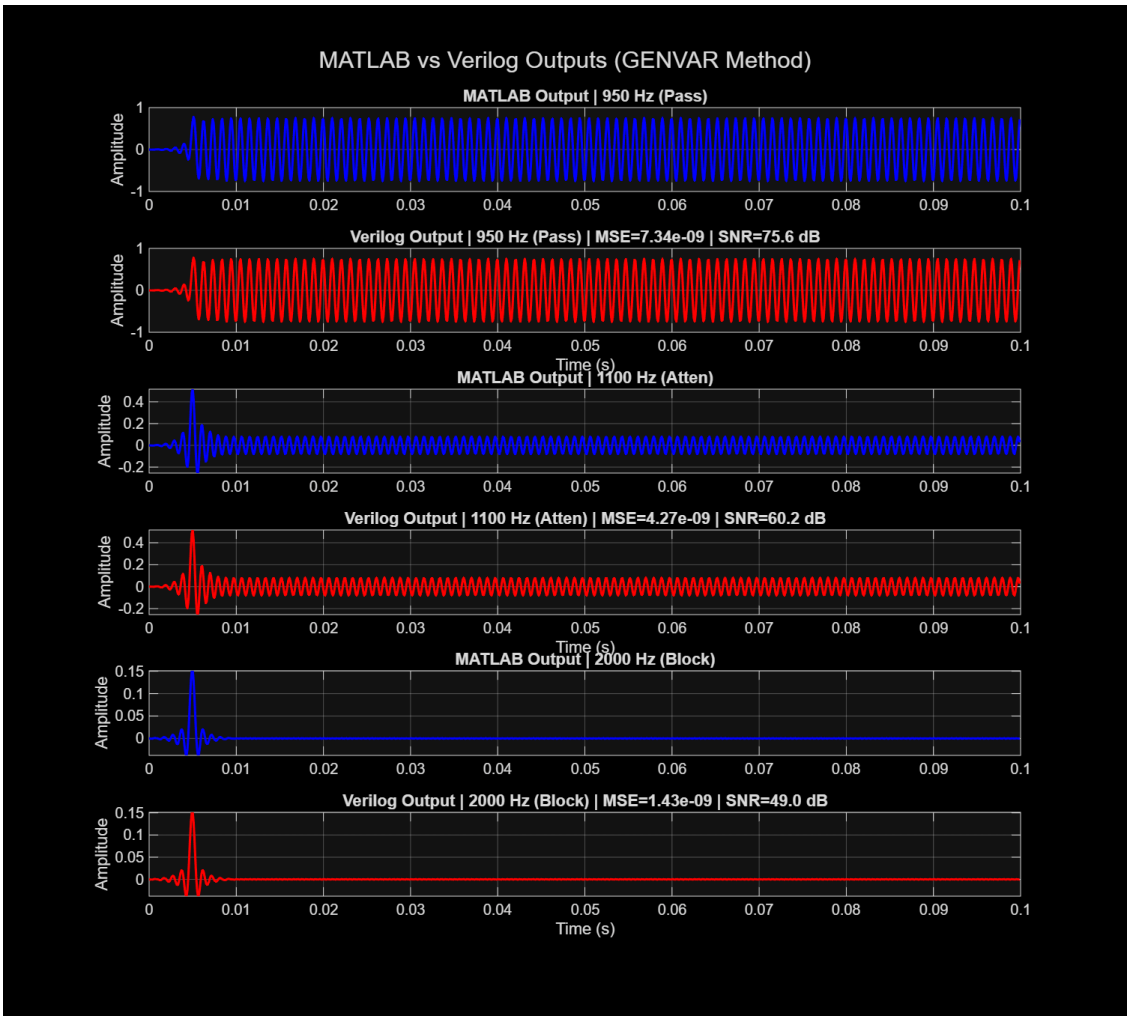

Figure 3: MATLAB vs Verilog comparison for the FIR filter implemented using the `genvar` construct in Verilog. The generate block automatically creates the required hardware structure for the FIR taps.

7 Conclusion

In this project we implemented FIR filters in Verilog using three different approaches.

- The direct form implementation follows the FIR equation directly but requires many multipliers.
- The generate/genvar implementation provides a modular and structured design using reusable blocks.
- The optimized implementation uses coefficient symmetry to reduce the number of multipliers by half.

These implementations demonstrate how digital filters can be designed efficiently in hardware using Verilog.